Assignment 1: MicroGrad
Scalar Automatic Differentiation Engine

John George Mikhael 23P0266

November 3, 20263/11/2026

Abstract

This report describes a scalar automatic differentiation engine and a small multilayer perceptron implemented without PyTorch or another machine-learning framework. The implementation represents each scalar as a node in a directed computational graph, records local derivative rules for elementary operations, and computes gradients by reverse-mode automatic differentiation.

1 Objective

The objective of this assignment is to understand the mechanics behind backpropagation. The implementation in `src/engine.py` provides a `Value` class for scalar data and gradients. The implementation in `src/nn.py` builds a simple neural-network API consisting of `Neuron`, `Layer`, and `MLP`.

2 Computational Graph

Each `Value` stores:

- `data`: the scalar value of the node;
- `grad`: the derivative of the final output with respect to the node;
- `_prev`: the set of parent nodes used to compute it;
- `_backward`: a closure containing the local chain-rule update.

For example, if $z = x + y$, then

$$\frac{\partial z}{\partial x} = 1, \qquad \frac{\partial z}{\partial y} = 1.$$

When the output gradient is $\partial L/\partial z$, the local backward function accumulates

$$\frac{\partial L}{\partial x} \mathrel{+}= \frac{\partial L}{\partial z}, \qquad \frac{\partial L}{\partial y} \mathrel{+}= \frac{\partial L}{\partial z}.$$

The use of += is essential when a value is reused in multiple branches of the graph.

3 Local Derivative Rules

The engine implements the following rules:

$$
\begin{aligned}
z &= x + y && \frac{\partial z}{\partial x} = 1, \quad \frac{\partial z}{\partial y} = 1, \\
z &= xy && \frac{\partial z}{\partial x} = y, \quad \frac{\partial z}{\partial y} = x, \\
z &= x^p && \frac{\partial z}{\partial x} = p x^{p-1}, \\
z &= \tanh(x) && \frac{\partial z}{\partial x} = 1 - \tanh^2(x), \\
z &= e^x && \frac{\partial z}{\partial x} = e^x.
\end{aligned}
$$

Subtraction, negation, and division are reduced to these primitives:

$$
x - y = x + (-y), \qquad -x = (-1)x, \qquad \frac{x}{y} = xy^{-1}.
$$

4 Topological Sort and Backpropagation

The graph is traversed by depth-first search. A visited set ensures that a shared node is visited once, while post-order insertion ensures that every parent appears before its dependent node in the resulting topological list. For a scalar output L, the global backward algorithm is:

1. Build the topological ordering starting at L.
2. Set $L.\texttt{grad} = 1$ because $\partial L/\partial L = 1$.
3. Traverse the ordering in reverse and invoke each node's local `_backward` closure.

This order guarantees that a node receives all gradient contributions from its children before its own gradient is propagated to its parents.

4.1 Diamond graph

Consider $c = a + a$ and $d = ca$. Since $d = 2a^2$,

$$
\frac{\partial d}{\partial a} = 4a.
$$

At $a = 3$, the correct result is 12. The implementation obtains this result because both paths from a are accumulated rather than overwritten.

5 Neural-Network API

A `Neuron` contains a list of randomly initialized weights and one randomly initialized bias. Given an input vector $\boldsymbol{x}$, it computes

$$
y = \tanh\left(\sum_i w_i x_i + b\right).
$$

A `Layer` contains multiple neurons and returns either a single `Value` for one output neuron or a list of values for multiple output neurons. An `MLP` creates consecutive layers using the requested size configuration, such as `MLP(2,[4,4,1])`.

The `parameters()` methods flatten all weights and biases so that a training loop can update them uniformly. The `zero_grad()` methods clear stored gradients before a new optimization step.

6 Training Demonstration

The accompanying notebook, `demo_micrograd.ipynb`, trains `MLP(2,[4,4,1])` on the four XOR examples:

$$(0,0) \mapsto -1, \quad (0,1) \mapsto +1, \quad (1,0) \mapsto +1, \quad (1,1) \mapsto -1.$$

The output neuron uses tanh, so the targets lie in $[-1, 1]$. The mean squared error is

$$\mathcal{L} = \frac{1}{N} \sum_{j=1}^{N} (\hat{y}_j - y_j)^2.$$

Each training step resets gradients, evaluates the loss, calls `backward()`, and applies manual gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}.$$

The notebook uses a fixed random seed for reproducibility and plots both the loss curve and the learned XOR predictions.

7 Testing and Results

The provided automated test suite contains 12 tests. It checks initialization, arithmetic operations, invalid operand handling, powers, activations, gradient accumulation, chain-rule behavior, division by zero, type casting, and the complete neural-network API. All 12 tests pass after implementation.

Table 1: Implemented components and validation

Component	Result
Scalar arithmetic and reverse operators	Implemented
Power, `tanh`, and `exp`	Implemented
Topological reverse-mode differentiation	Implemented
Neuron, Layer, and MLP	Implemented
Automated tests	12 passed

8 Design Justifications

- Scalars are represented as graph nodes so each operation can retain both its forward result and local derivative rule.
- Parent nodes are stored in a set because graph membership is what is needed for traversal; gradient contributions are accumulated inside the backward closures.
- Reverse-mode differentiation is appropriate because the network has one scalar loss and many parameters.
- The neural-network API intentionally stays small: parameter collection and gradient reset are sufficient for a transparent hand-written optimization loop.

9 Conclusion

This assignment demonstrates that backpropagation can be implemented with basic Python objects, local derivative formulas, and a topological traversal. The same chain rule used by this small engine is the foundation of gradient computation in larger deep-learning frameworks.